INDEPENDENT VERIFICATION REPORT

KLB SeedChain GPU 0.2.0

What works, what does not, and where it is practically useful

Human-readable / ELI5 edition | RTX 5070 Ti Laptop | 16 August 2026

BOTTOM LINE The 207x demo compression is real for the supplied reconstructible sequence. The GPU implementation is correct and very fast. The same ratio does not carry over to arbitrary motion: the generic fitter became larger than dense float3 on a challenging 16-frame deformation test.

DEMO STORAGE	FULL GPU CHECK	DIRECT QUERY
207.321x	**0 lineage mismatches**	**0.041 ms**

ELI5 analogy: keep one master LEGO model, a tiny recipe card for each animation frame, and a short bag of replacement bricks only when something unpredictable changes.

1. The answer in plain English

VERDICT PASS for generated or strongly predictable point sequences. CONDITIONAL for real captured animation. FAIL as a claim of universal 200x geometry compression.

The seed-chain idea works when most frames can be recreated from a known recipe. Instead of storing every point at every time step, it stores one compact starting shape, 96 bytes of instructions per frame, and sparse 16-byte correction records. That is why the supplied 240-frame chain shrinks from 188,743,680 bytes of dense float3 data to 910,392 bytes.

The same trick is not magic compression. If almost every point needs a correction every frame, the correction log becomes larger than the dense sequence. That happened in the generic fitting challenge: 93.16% of point-frames needed novelty records, making the accurate chain 27% larger than dense float3.

What the implementation genuinely solves

- Stores a long, reconstructible point sequence in a tiny container.
- Rebuilds any selected frame directly on an RTX 5070 Ti Laptop GPU.
- Queries compressed points without first materializing the complete dense sequence.
- Produces the same compact point/lineage event set as its materialized-frame path.
- Keeps unpredictable changes as sparse, checkpointed novelty records.

What it does not solve

- It is not a universal 200x compressor for arbitrary animation.
- It stores point positions only: no faces, normals, UVs, materials, or changing topology.
- It does not prove physical Klein-bottle memory, zero heat, or zero bandwidth.
- Its FNV integrity chain detects accidental change but is not a cryptographic signature.

2. Test environment and scope

Item	Measured configuration
GPU	NVIDIA GeForce RTX 5070 Ti Laptop GPU, compute capability 12.0
Driver / runtime	NVIDIA driver 591.59; CUDA driver API 13.1; runtime 12.8
Compiler	CUDA 12.8.61, MSVC 19.44, native sm_120 plus compute_120 PTX
Device	46 SMs, 36 MiB reported L2, 192-bit memory bus, 11.94 GiB VRAM

Item	Measured configuration
Power context	Windows Balanced plan, AC power; no fixed vendor performance profile recorded

3. How SeedChain works (ELI5)

Imagine a flip-book containing 240 pictures of the same object:

1. Store one compact master model. Each point uses a 37-bit log-spherical record.
2. Store a recipe card for each picture. A 96-byte node contains the seed, motion predictor, grammar state, checkpoint information, and hash links.
3. Store only surprises. A 16-byte novelty record is written when a point cannot be predicted closely enough.
4. Jump to any frame. The GPU starts from the master, applies that frame's recipe, then walks back no farther than the nearest checkpoint to collect corrections.
5. Ask a local question. The query tests cone/radius support, route compatibility, and an SDF guard, then appends only verified events.

WHY 207x IS POSSIBLE A dense sequence repeats 65,536 coordinates 240 times. SeedChain stores the base once and explains almost all later motion procedurally. Only 0.232054% of point-frames need novelty records.

Storage equation

KLSC1 bytes = 256-byte header + 96 x frames + 16 x novelty records + embedded base words.

Dense float3 bytes = 12 x points x frames.

For large sequences, the approximate break-even novelty density is below 75%. Useful compression requires much lower density: about 37.5% for 2x and 7.5% for 10x, before header/base overhead.

4. Compression evidence

Case	Frames	Container	vs float3	Novelty
Included generated	240	910,392 B	207.321x	0.2321%
Bunny + generated motion	240	508,856 B	203.451x	0.2308%
Bunny generic fit, 0.2% max error	16	8,741,384 B	0.7896x	93.1562%
Bunny generic fit, 4.0% max error	16	4,061,768 B	1.6992x	42.3043%

The Bunny tests preserve vertex positions only. The 240-frame Bunny chain still uses generated motion; it is not a fitted real scan sequence.

5. The generic fitter reality check

Sixteen frames were exported from the Bunny-based generated chain and then treated as an unknown external sequence. The fitter was allowed only one global uniform scale, one Y-axis rotation, and translation per frame. This deliberately tests whether the current generic predictor can explain complex per-point motion without knowing the original grammar.

Residual cutoff	Novelty density	Compression	RMS/radius	Max/radius
0.2%	93.1562%	0.7896x	0.0334%	0.2000%
0.4%	92.8466%	0.7921x	0.0516%	0.4000%
1.0%	89.6210%	0.8201x	0.2475%	1.0000%
2.0%	77.0262%	0.9511x	0.7075%	2.0000%
4.0%	42.3043%	1.6992x	1.7981%	4.0000%

INTERPRETATION The fitter is accurate, but its predictor is too simple for this deformation. At the useful 0.2% maximum-error setting, storing corrections is more expensive than storing float3. Compression appears only after accepting a much larger 4% maximum positional error.

The independent verify-sequence command confirmed 0.033386% RMS and 0.199999% maximum error for the accurate fit. This proves position reconstruction against the source PLYs. It does not yet prove that event membership versus the original source is unchanged.

6. GPU performance

Case	Depth	Seed query	Dense query	Penalty	Events
Included, prescribed run	15	0.04124 ms	0.01325 ms	3.11x	79
Included, full-point verify	15	0.04101 ms	0.01230 ms	3.33x	79
Checkpoint frame 16	0	0.01343 ms	0.01181 ms	1.14x	55
Bunny generated	15	0.02672 ms	0.01211 ms	2.21x	2,800
Bunny fitted, accurate	15	0.05128 ms	0.01249 ms	4.10x	2,761
100% event yield	15	0.04105 ms	0.01309 ms	3.14x	65,536

Repeatability at final frame (30 fresh processes)

Mode	p50	p95	p99	Mean
Decode one frame	0.039165 ms	0.039762 ms	0.039858 ms	0.039226 ms
Direct seed query	0.041012 ms	0.043019 ms	0.043067 ms	0.041311 ms
Dense query	0.011993 ms	0.013468 ms	0.015226 ms	0.012259 ms

7. What the timings mean

The direct compressed path is slower than querying a frame that is already materialized. That is expected: it decodes 37-bit records, runs grammar/topology math, and searches up to 16 novelty ranges before evaluating the same event test.

ONE QUERY VS MANY At frame 239, one direct query costs about 0.041 ms. Decode-then-query costs about 0.039 + 0.012 = 0.051 ms, so direct wins for a one-off question. By roughly the second query against the same frame, materializing once and reusing the dense frame becomes faster, if the extra memory is acceptable.

Checkpoint depth matters. Direct query time rose from roughly 0.013-0.015 ms at depth 0-1 to 0.039-0.043 ms at depth 15. This is the cost of dependent novelty lookups. Smaller checkpoint strides trade more file storage for faster random access.

A 200,000-launch seed-query stress run sustained 0.041111 ms and 1.594 billion candidate points/s. After ramp-up, the GPU reported 98-99% utilization, about 128.6-130.6 W, and 63-69 C. Coarse nvidia-smi memory utilization rounded to 0%, consistent with this sub-1 MiB chain fitting in cache, but it is not a DRAM counter.

8. Correctness evidence

- Package manifest: every listed file matched SHA-256; supplied KLSC chain hash matched the manifest.
- CPU oracle suite: 1 of 1 test executable passed, covering bit packing, parity, swizzle, Klein seam rules, PLY I/O, checkpoints, hashes, fitting, and error reporting.
- Full GPU comparison: all 65,536 points checked at frame 239; zero lineage mismatches and numerical error below the benchmark tolerance.
- Event equivalence: all 79 default events matched point identity and lineage; SDF and guard differences printed as 0.000000.
- Route partition: route 0 produced 38 events and route 1 produced 41, exactly partitioning the 79-event unfiltered set.
- High-yield path: 65,536 of 65,536 points appended; every compressed event matched the dense event set.
- Compute Sanitizer: zero memory-access errors across decode, direct query, dense query, and verification.
- CUDA compiler: native sm_120 kernels built with zero register spills; seed/decode uses 40 registers and dense query uses 18.

Important precision note: the benchmark prints CPU/GPU error with only three decimals. A displayed 0.000 means 'below the printed precision and acceptance threshold,' not necessarily bit-for-bit identical coordinates.

9. Practical use cases

BEST MATCH Use SeedChain where motion is mostly known and deterministic, edits are sparse, stable point identity matters, and you need occasional local GPU queries without keeping every dense frame resident.

Procedural vegetation, particles, and branching effects

Strong fit. Wind, growth, branching, and repeated cycles can be regenerated; artist edits become novelty records. Add cluster-level predictors for large assets.

Rollback/replay and deterministic simulation logs

Strong fit when every participant consumes the same chain file. Store closed dynamics as seeds and only retain external interventions.

Sparse spatial event or culling service

Strong fit for one-off frame queries that return few candidates: collision candidates, proximity alarms, local support checks, or render culling.

Repeated mechanical motion and digital twins

Promising for rigid assemblies, conveyors, robot cells, and cyclic machines. Per-part rigid predictors are the obvious next extension.

Stable-correspondence LiDAR/depth sequences

Conditional. Useful when scans are registered and most motion is rigid or cluster-rigid. Poor fit when correspondence changes, occlusion is heavy, or points are resampled.

Network or disk streaming to memory-limited GPUs

Promising when sending one base plus compact nodes is cheaper than streaming dense frames. Chain segments can be prefetched and queried directly.

Poor matches

- Cloth, fluids, crowds, or arbitrary deformation with the current single global predictor.
- Topology-changing meshes, unstable vertex correspondence, remeshing, or particle birth/death without an identity layer.
- Full production meshes requiring faces, normals, UVs, materials, skinning, or blendshape metadata.
- Workloads that query the same frame many times and can afford to materialize it once.
- Security/provenance systems that require cryptographic authentication rather than FNV integrity checks.

10. Limitations and issues found

Universal compression is unproven: The measured 207x applies to a reconstructible generated sequence. The generic fitting challenge was 0.79x at the accurate setting.

Dense-reference comparison is decode consistency: The GPU benchmark compares direct reconstruction with a dense frame materialized from the same compressed chain. It does not compare event membership against the original source PLY sequence.

Hardware counters unavailable: Nsight Compute was installed, but Windows denied performance-counter access with ERR_NVGPUCTRPERM. DRAM bytes, L2 hit rate, occupancy, stalls, and atomic serialization therefore remain unmeasured.

Cross-compiler regeneration is not byte-identical: The supplied GCC-built chain and an MSVC regeneration had identical size/statistics but different terminal/SHA hashes. Two MSVC regenerations matched each other. Share the built chain or adopt canonical math for cross-platform deterministic generation.

Windows helper can report false success: The supplied PowerShell build script continued after CMake failed on a stale CUDA 12.9 Visual Studio integration and ultimately exited 0. Pinning CUDA 12.8 in CMake produced a valid native build.

One-bit route is not identity: Durable identity still requires point/base address, ordered node history, schema version, and novelty records.

11. Recommended next engineering steps

1. Add a source-frame event oracle so fitted compression is accepted only when event membership/order also matches the original sequence.
2. Replace one global Y-similarity predictor with per-cluster rigid/quaternion predictors and per-cluster quantization bounds.
3. Sweep checkpoint strides 4, 8, 16, 32, and 64 on each real workload; jointly report file size, p95 latency, and novelty density.
4. Enable NVIDIA performance counters and collect real DRAM/L2, occupancy, warp-stall, instruction, and atomic metrics.
5. Make procedural generation cross-platform deterministic using canonical/fixed approximations or by distributing authoritative node data.
6. Fix the Windows build helper to stop on non-zero native exit codes and optionally select an installed CUDA toolkit explicitly.
7. Compare against conventional point-cloud, geometry, and video codecs at equal error and required random-access behavior.
8. Add stable face/index streams only for applications that truly need mesh output; keep the direct event path position-only when that is sufficient.

GO / NO-GO RULE Proceed when novelty stays sparse, maximum error is below the application guard margin, source-event classifications are preserved, and avoided storage/transfer is worth the 1.1x-4.1x direct-query compute penalty. Otherwise materialize, segment, or use a conventional codec.

Appendix A. Evidence and commands

Primary measured artifacts created in the project directory:

- seedchain_results.csv - prescribed 20-repeat final-frame run.
- seedchain_depth_sweep.csv - checkpoint-depth sweep across nine frames.
- seedchain_process_runs.csv - 30 fresh-process stability runs.
- seedchain_full_verify_results.csv - all 65,536 points verified.
- bunny_seedchain_results.csv - real Bunny geometry with generated motion.
- fitted_sequence_results.csv - accurate and loose generic-fit GPU runs.
- seedchain_high_yield.csv - 100% event-yield compaction test.

Core commands

Purpose	Command summary
Build	cmake -S . -B build-cuda128 -G "Visual Studio 17 2022" -A x64 -T "cuda=C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.8" -DKLB_CUDA_ARCH=120 -DKLB_REQUIRE_CUDA=ON
CPU tests	ctest --test-dir build-cuda128 -C Release --output-on-failure
Inspect	klb_seedchain.exe inspect data\procedural_65536_240f.klsc
GPU verify	klb_seedchain_bench.exe data\procedural_65536_240f.klsc --frame 239 --mode all --repeats 200 --verify 65536
Sanitizer	compute-sanitizer --tool memcheck klb_seedchain_bench.exe ...
Fitter verify	klb_seedchain.exe verify-sequence bunny_frames_16.txt bunny_fitted_16f_thr002.klsc

Evidence boundary

The report is based on the supplied package, locally generated test artifacts, and direct execution on the stated laptop. The Stanford Bunny adapter uses vertices only. No claim is made about faces, materials, visual perceptual quality, adversarial integrity, or source-level event preservation beyond the tests described.

Overall assessment: the architecture is a credible specialized seed-plus-novelty event substrate. Its value comes from matching the right predictable workload, not from treating the demo compression ratio as universal.